visualize_decision_quality

February 12, 2026

1 Figure 13. (a) Plan quality of LlaMA2-13B as resource expands

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

DATA_PATH = "../results/micro_scopic.csv"

# Define styles similar to other notebooks
policy_map = {
    "suika": "Suika",
    "dp": "Sia/Suika-DP",
    "rubick": "Rubick",
    "metis": "Suika-Metis",
}

policies = ["Suika", "Sia/Suika-DP", "Rubick", "Suika-Metis"]

def read_data():
    with open(DATA_PATH, 'r') as f:
        lines = f.readlines()
    data = {
        '# GPUs': [16, 24, 32, 40, 48, 56, 64],
        'base_iter_time': {}
    }
    for line in lines:
        policy, n_gpu, iter_time = line.strip().split(', ')
        policy_name = policy_map[policy]
        if policy_name not in data:
            data[policy_name] = ['-'] * len(data['# GPUs'])
            data['base_iter_time'][policy_name] = None
        iter_time = float(iter_time)
        n_gpu = int(n_gpu)
        if n_gpu == data['# GPUs'][0]:  # base case for 16 GPUs
            data['base_iter_time'][policy_name] = iter_time

        base_iter_time = data['base_iter_time'][policy_name]
```

```
        index = data['# GPUs'].index(n_gpu)
        data[policy_name][index] = f"{base_iter_time/iter_time:.3f}x"

    del data["base_iter_time"]
    print(data)

    return data

data = read_data()


df = pd.DataFrame(data)

# Process the dataframe to handle '-' and '*'
for col in df.columns:
    if col != '# GPUs':
        # Remove 'x' and convert to numeric, coercing errors
        df[col] = pd.to_numeric(df[col].str.replace('x', ''), errors='coerce')
        # Forward fill NaN values from '-', '*'
        df[col] = df[col].ffill()

print("Processed Data:")
print(df)


policy_styles = {
    "Suika":  {"color": "#34495e", "hatch": "", "linestyle": "-", "marker":␣
 ↪"o"},
    "Sia/Suika-DP":    {"color": "#e67e22", "hatch": "/", "linestyle": "--",␣
 ↪"marker": "s"},
    "Rubick": {"color": "#f1c40f", "hatch": "x", "linestyle": "-.", "marker":␣
 ↪"^"},
    "Suika-Metis":   {"color": "#3498db", "hatch": ".", "linestyle": ":",␣
 ↪"marker": "d"},
}
```

```
{'# GPUs': [16, 24, 32, 40, 48, 56, 64], 'Suika': ['1.000x', '1.400x', '1.750x',
'1.989x', '2.303x', '2.500x', '2.612x'], 'Suika-Metis': ['1.000x', '1.346x',
'1.750x', '2.108x', '-', '-', '-'], 'Sia/Suika-DP': ['1.000x', '-', '1.522x',
'-', '-', '-', '-'], 'Rubick': ['1.000x', '-', '1.306x', '-', '-', '-',
'1.683x']}
Processed Data:
   # GPUs  Suika  Suika-Metis  Sia/Suika-DP  Rubick
0      16  1.000        1.000         1.000   1.000
1      24  1.400        1.346         1.000   1.000
2      32  1.750        1.750         1.522   1.306
3      40  1.989        2.108         1.522   1.306
```

```
4       48  2.303         2.108          1.522    1.306
5       56  2.500         2.108          1.522    1.306
6       64  2.612         2.108          1.522    1.683
```

1.0.1 Line Plot: Throughput vs. Number of GPUs

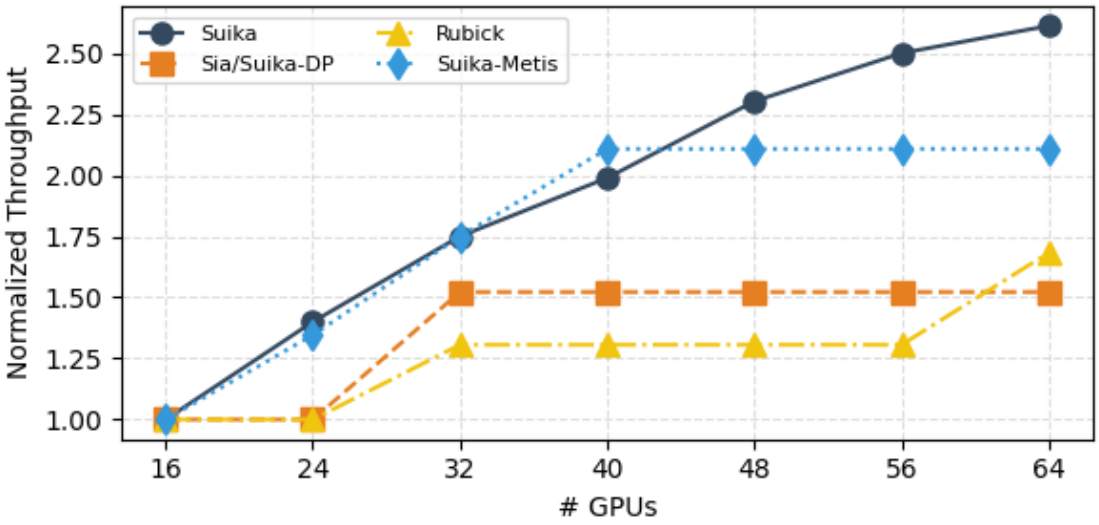
This plot shows how the throughput of each policy scales with the number of GPUs. The x-axis represents the number of GPUs, and the y-axis represents the throughput. This visualization makes it easy to see the scaling trend of each policy.

[2]:
```
# Solution 2a: Compact Line Plot with large markers
from tkinter import font


plt.figure(figsize=(6, 3))

for policy in policies:
    plt.plot(df['# GPUs'], df[policy],
             label=policy,
             color=policy_styles[policy]['color'],
             linestyle=policy_styles[policy]['linestyle'],
             linewidth=1.5,
             marker=policy_styles[policy]['marker'],
             markersize=8) # Larger markers

plt.xlabel('# GPUs')
plt.ylabel('Normalized Throughput')
# plt.title('Throughput Scaling of Llama2-13B under Different Policies',␣
 ↪fontsize=10)
plt.xticks(df['# GPUs']) # Set X ticks strictly to the #GPUs values
plt.legend(fontsize=8, loc='upper left', ncol=2)
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
import os
fig_dir = "../fig"
os.makedirs(fig_dir, exist_ok=True)
plt.savefig(f'{fig_dir}/decision_quality.pdf')
plt.show()
```


Suika
Sia/Suika-DP
Rubick
Suika-Metis
Normalized Throughput
2.50
2.25
2.00
1.75
1.50
1.25
1.00
16
24
32
40
48
56
64
GPUs